

《嵌入式系统设计》实验指导书

——SP32 FreeRTOS 综合实验



上海交通大学机械与动力工程学院
基础实验与创新实践教学中心

2025.11

实验一：带 WiFi 通信功能的数字时钟

1 实验目的

1. 学习 ESP32 WiFi 模块的初始化配置，掌握 STA 模式下的网络连接技术。
2. 理解 TCP/IP 协议基础，实现嵌入式设备与 PC 机的 TCP 客户端 - 服务器通信。
3. 掌握 FreeRTOS 实时操作系统的多任务编程，理解任务创建及优先级分配、信号量互斥锁保护机制。
4. 初步了解 ESP32 的硬件定时器（GPTimer）的配置和使用方法，理解周期性中断与计数原理。
5. 培养系统模块化编程思想，理解各功能模块的相互调用。

2 任务要求

1. 基于 ESP32 硬件定时的计时功能：调用 ESP32 中的时钟源，通过回调函数实现数字时钟计时。
2. 网络远程控制与实时状态监控：上位机可以通过 WiFi 向开发板发送指令（START、STOP、RESET），实现下位机开发板的时钟计时，同时并将时钟时间实时的向上位机发送。

3 项目框架介绍

本示例项目基于 ESP-IDF 开发，使用 FreeRTOS 实现 WiFi 双向通信和硬件计时。系统使用 esp32 硬件定时产生中断，通过回调函数执行定时任务，同时利用队列（Queue）实现网络指令的传递。项目核心模块包括：

1. 计时模块（counter.c）

`gptimer_init()`：配置 1MHz 分辨率的向上计数器，设置 1 秒周期的闹钟中断

`TimerCallback()`：定时器中断回调函数，设置计时标志位

`counter_start()`：计数任务函数，根据标志位更新时间（时 / 分 / 秒）

2. 网络模块（wifi.c）

`init_wifi()`：初始化 WiFi 为 STA 模式，连接指定无线网络

`tcp_server_task()`：创建 TCP 服务器（端口 3333），处理客户端连接

`tcp_send_data_task()`：定时向客户端发送当前时间数据

`tcp_recv_cmd_task()`：接收并解析客户端指令（START/STOP/RESET）

3.1 任务设定及优先级分配

系统包含以下主要任务，按优先级从高到低排列：

1、硬件计时任务 (`counter_start()`，优先级 4)

系统核心功能任务，通过硬件定时触发中断，进行精准定时。

2、TCP 数据发送任务 (`tcp_send_data_task()`，优先级 3)

Esp32 实时向上位机发送计时的时间信息。

3、TCP 命令处理任务 (`tcp_recv_cmd_task()`，优先级 2)

从队列 `tcp_command_queue` 中取出指令并执行相应操作，修改系统状态变量，控制硬件计时任务的启停。

4、TCP 服务器任务 (`tcp_server_task()`，优先级 1)

承担了网络连接管理任务，同时实时监听客户端连接，接收上位机指令数据，解析指令并通过队列 `tcp_command_queue` 发送给命令处理任务。

3.2 GUI

本项目使用 PyQt5 开发了一个上位机 GUI 界面，提供了 ESP32 网络 IP 地址和 TCP 服务端口号设置控件、计时器启停以及复位按钮、时钟显示盘、系统状态信息打印框等。GUI 界面截图 2 如所示。

运行 Python 脚本有两种方法，方法一：在 VSCode 中打开终端（图 3），输入 `python ./GUI/send_receive.py`，点击 Enter，即可运行 GUI 界面；方法二：在 VSCode 中下载 Python 扩展插件，安装完毕后，该 GUI 的脚本路径为 `./GUI/send_receive.py`，在 VSCode 中打开该脚本，如图 1 点击右上角的运行按钮即可打开 GUI 界面。

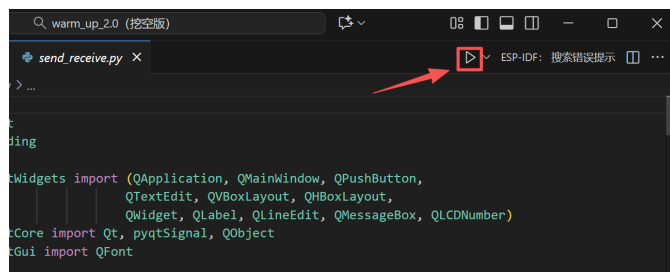


图 1



图 2

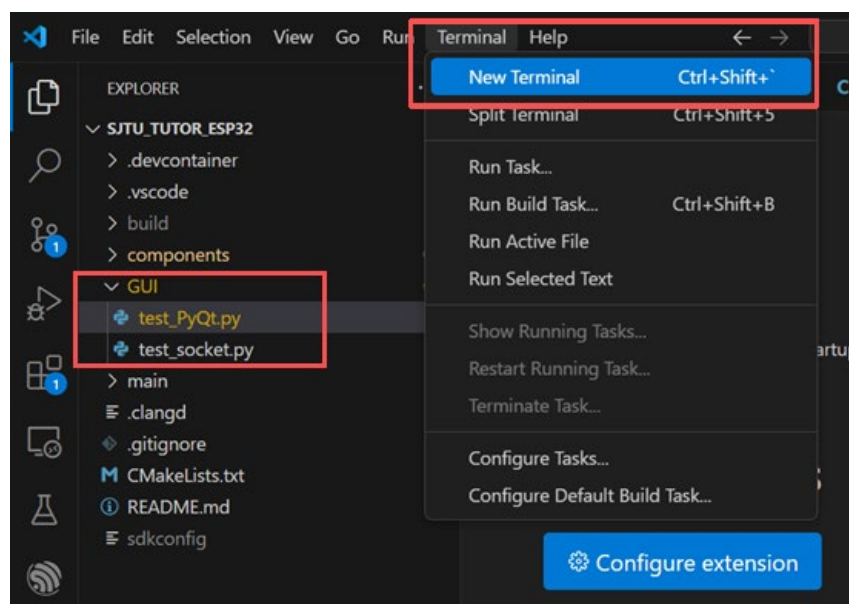


图 3

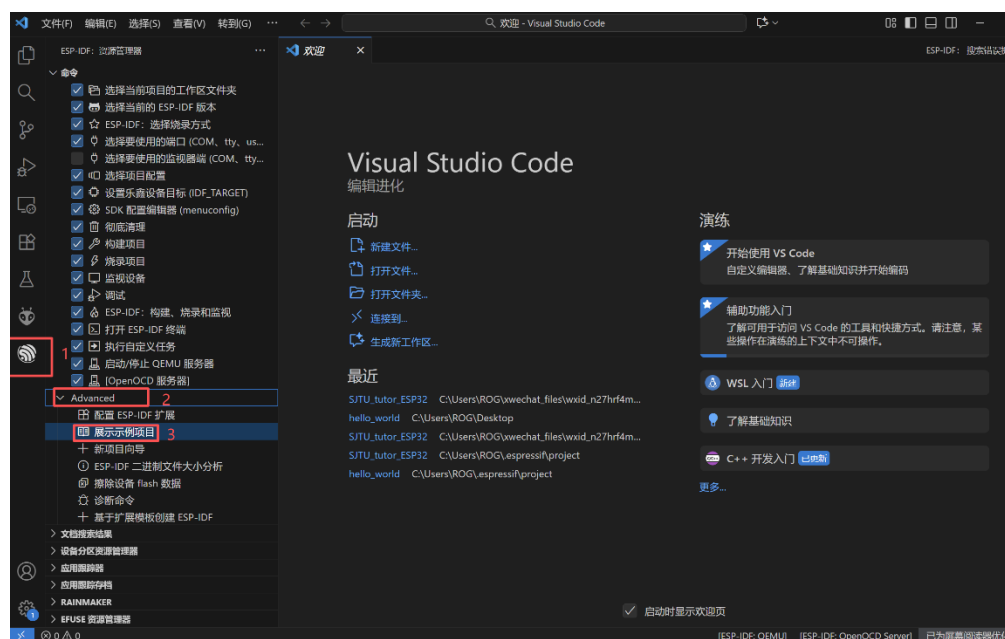
4 实验步骤指导

4.1 环境安装与测试

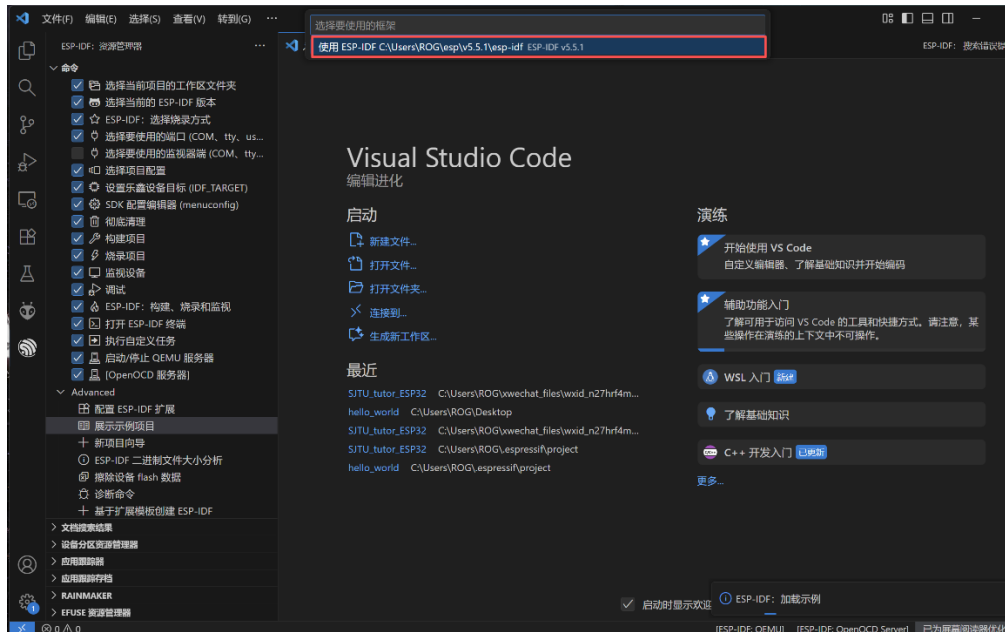
本实验已为同学们安装好 VSCode 与 ESP-IDF，同学们只需要打开 VSCode 进行环境测试。步骤如下：

(1) 打开电脑的“Visual Studio Code”。

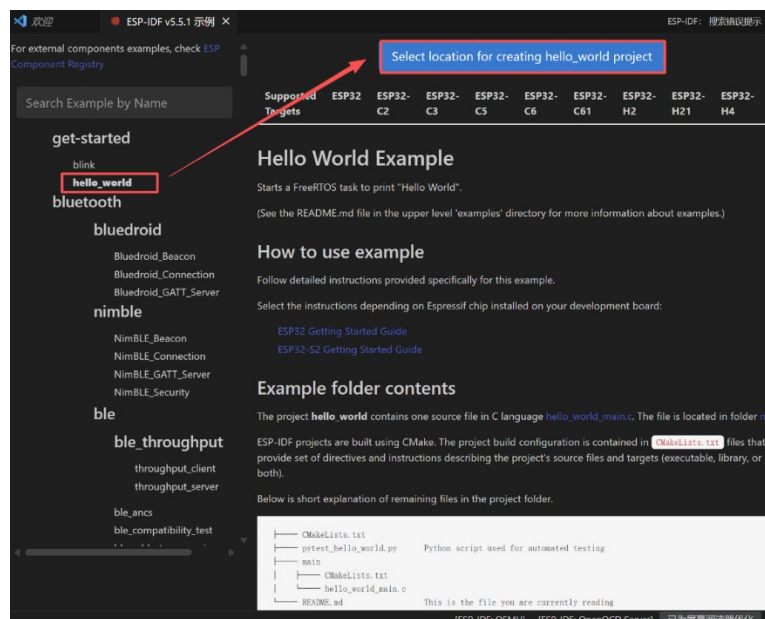
首先打开 ESP—IDF 资源管理器，找到“展示示例项目”并点击



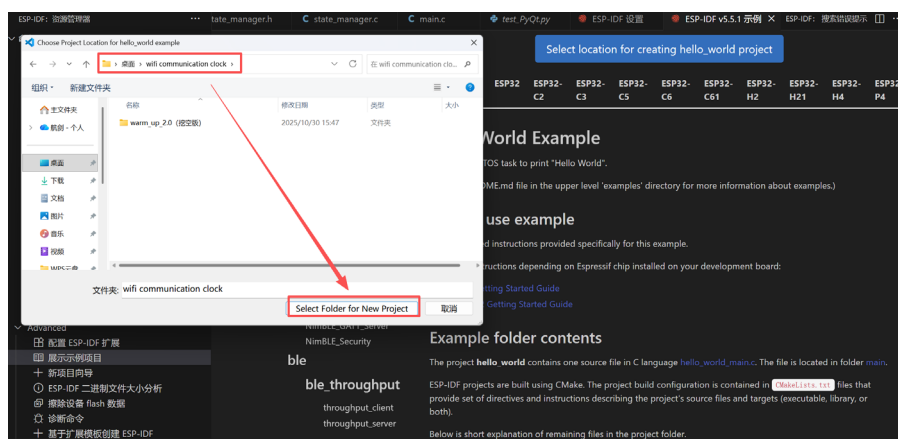
选择使用 ESP-IDF 框架



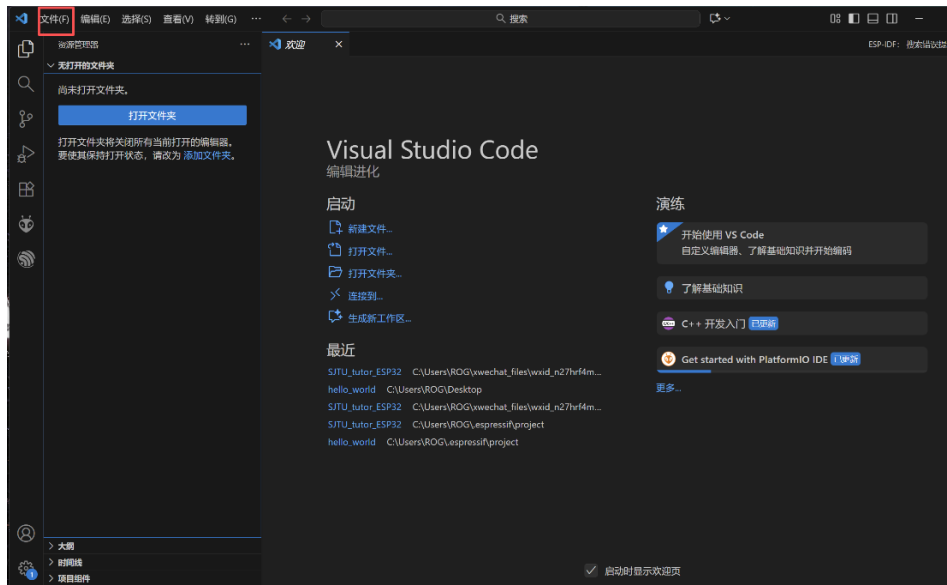
选择“Hello World”示例项目，下载到指定文件路径下



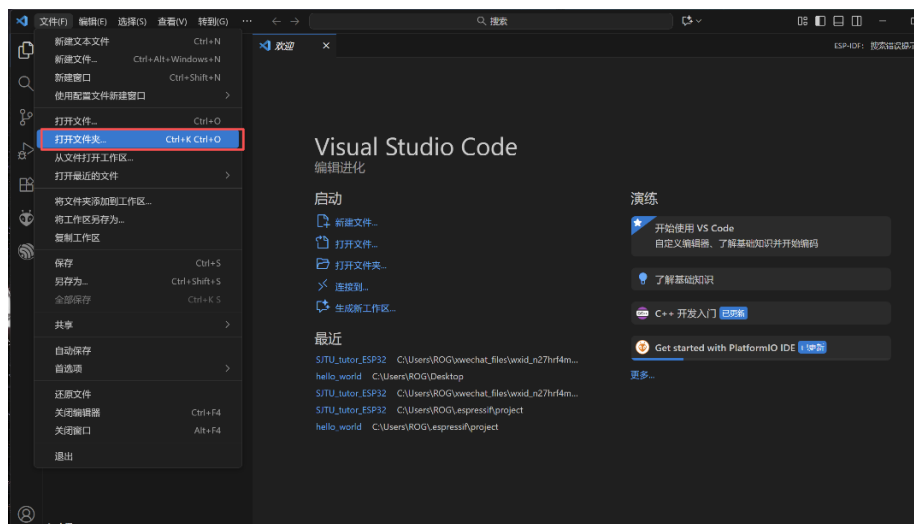
选择好下载路径后（最好路径中不要出现中文），点击“Select Folder for New Project”。



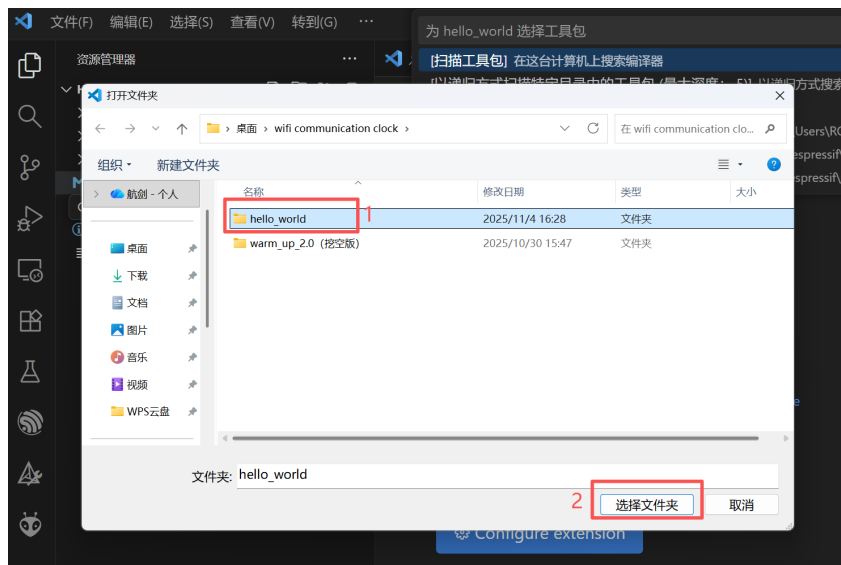
到此我们已将示例项目下载到了我们指定的文件夹，下面我们运行示例项目检查环境是否配置成功，点击文件。



点击“打开文件夹”

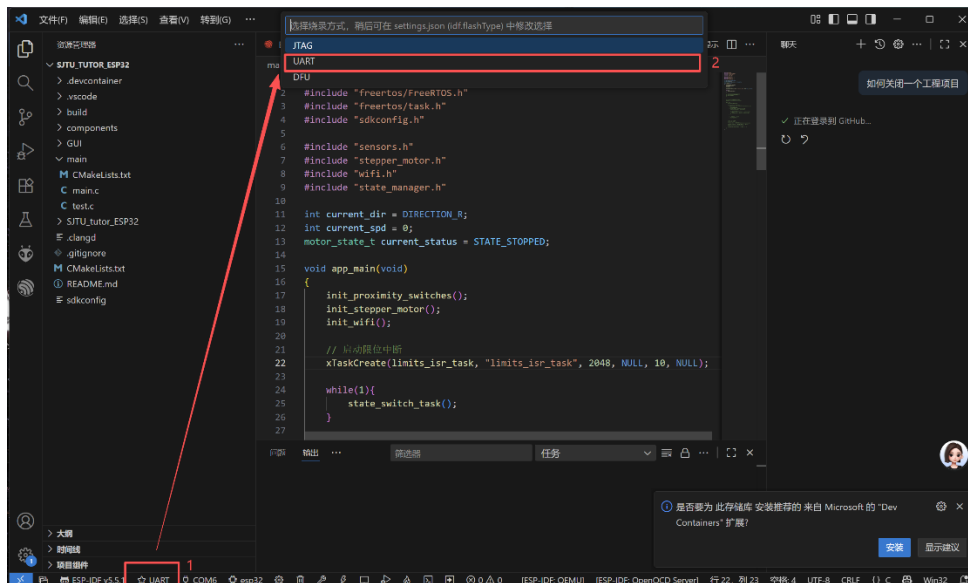


选择我们上一步安装的“Hello world”示例项目文件夹，并打开。

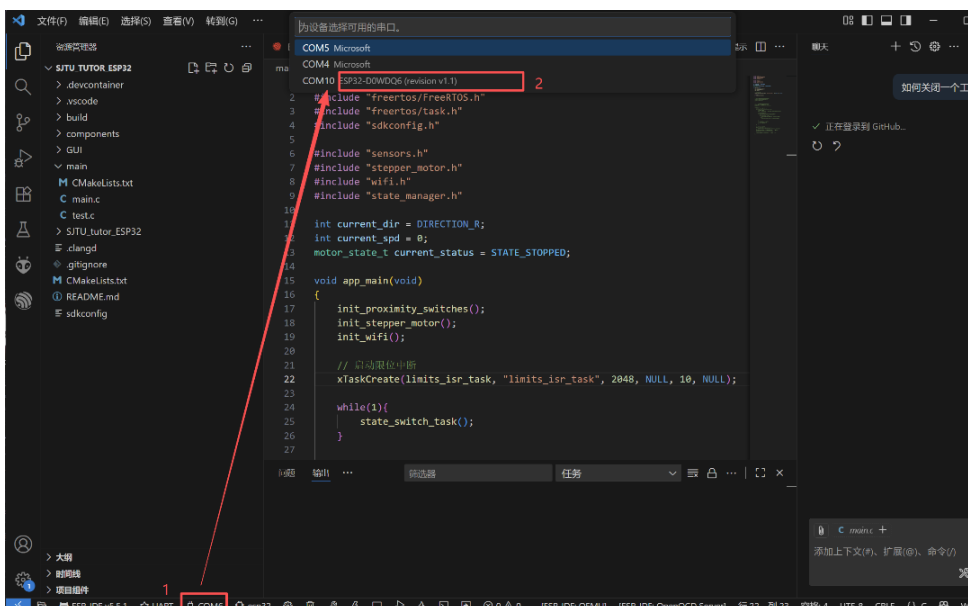


实验一 带 WiFi 通信功能的数字时钟

(2) 将电脑与 Esp32 开发板用数据线连接，打开示例项目后，下面进行示例项目的编译与烧录，选择烧录方式为“UART”。

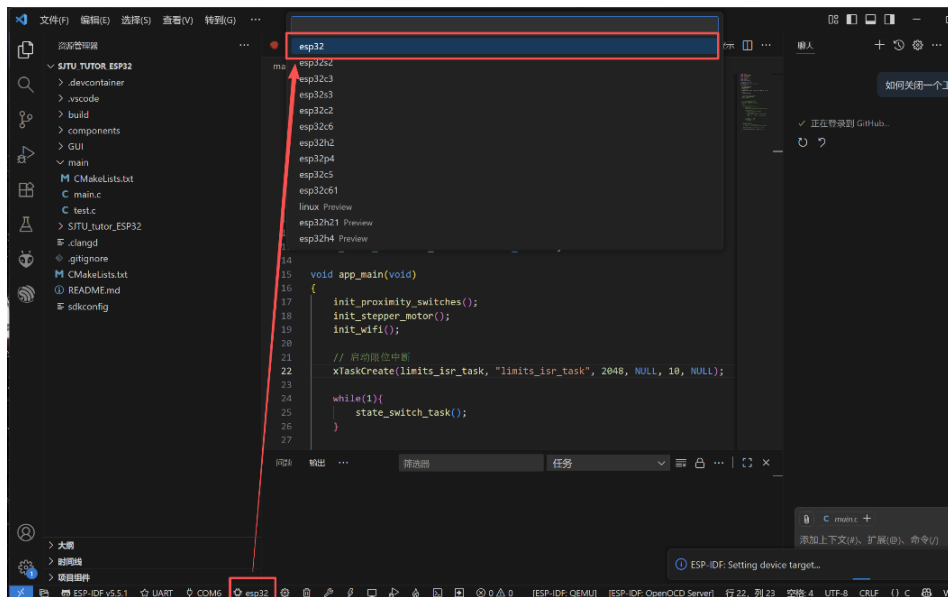


选择串口，选择带有“ESP32”芯片标识的串口

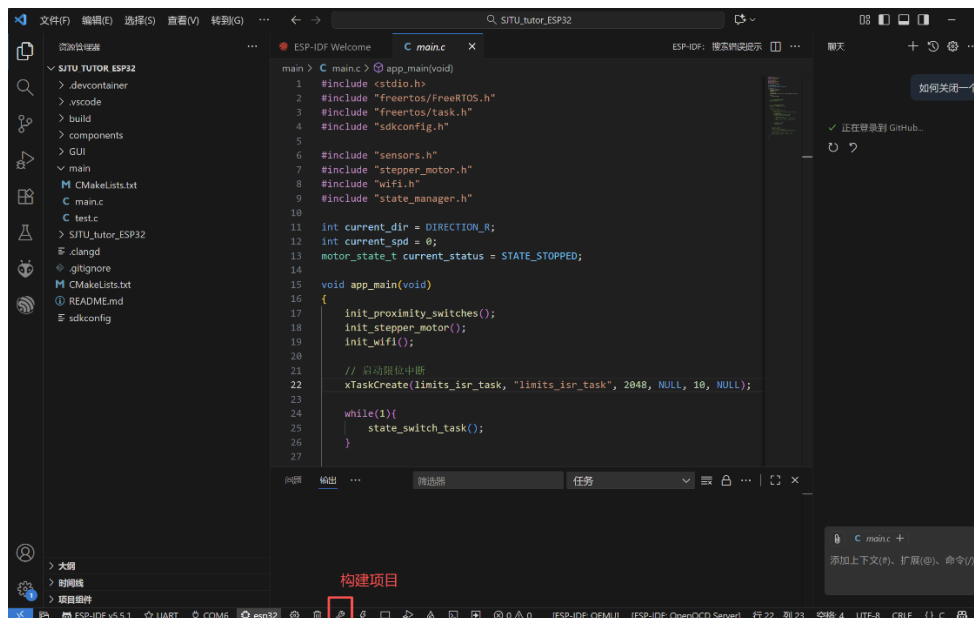


选择“ESP32”芯片

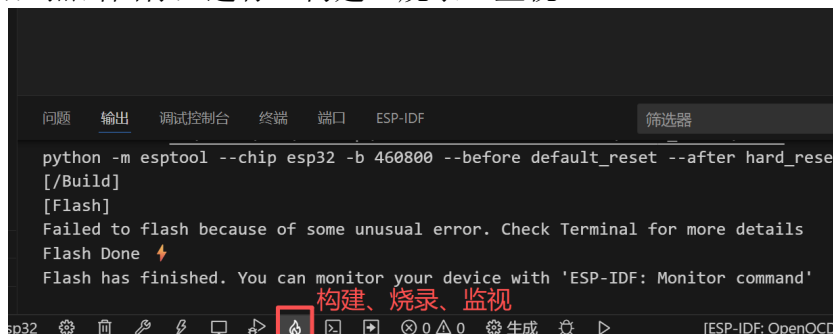
实验一 带 WiFi 通信功能的数字时钟



点击“构建项目”，进行编译



构建成功后，点击图标，进行“构建、烧录、监视”

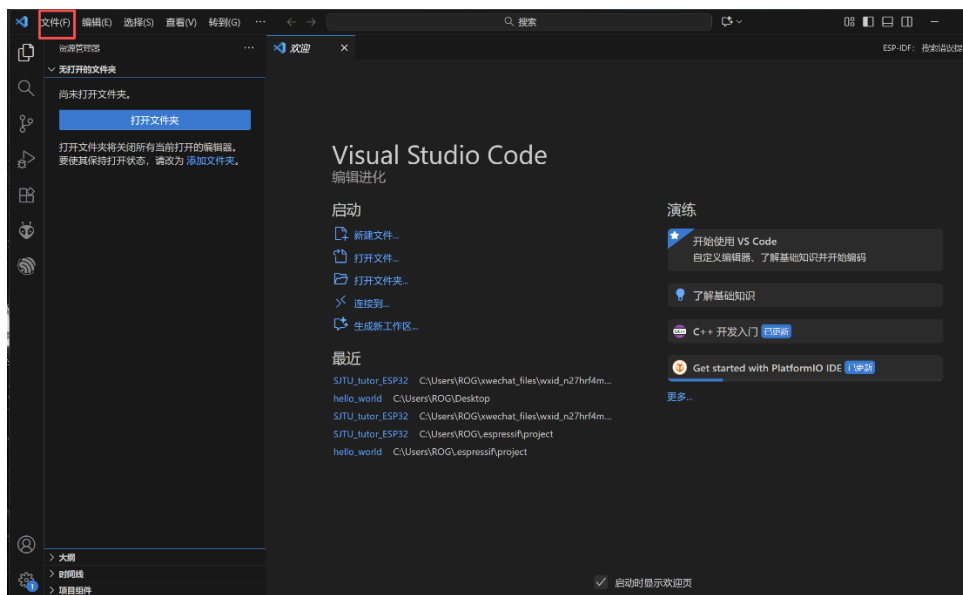


若环境配置正确，在监视窗口我们可以看到每隔一段时间会打印出“Hello World”。

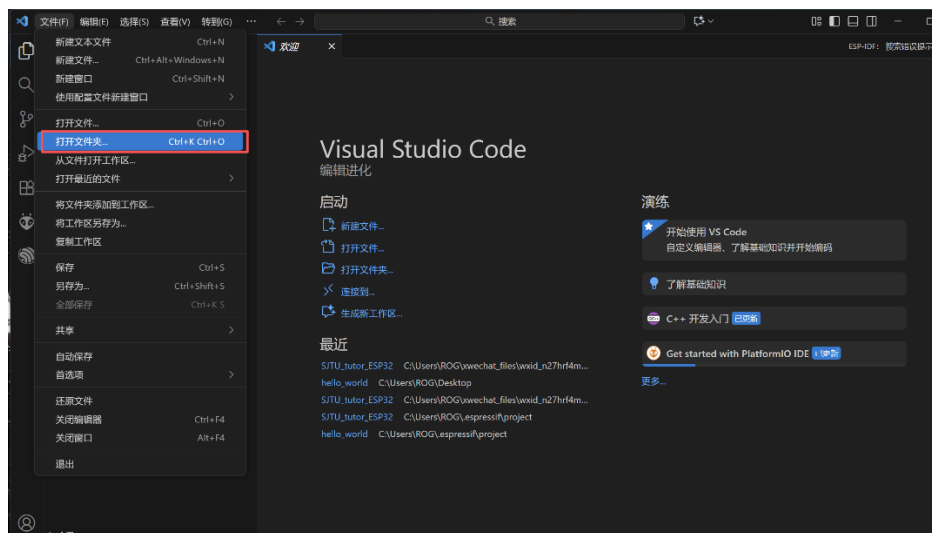

```
I (241) spi_flash: flash io: dio
W (244) spi_flash: Detected size(4096k) larger than the size in the binary image header.
I (257) main_task: Started on CPU0
T (267) main_task: Calling app_main()
Hello world!
This is esp32 chip with 2 CPU core(s), WiFi/BT/BLE, silicon revision v1.1.
Minimum free heap size: 306072 bytes
```

4.2 打开 WiFi 数字时钟示例项目

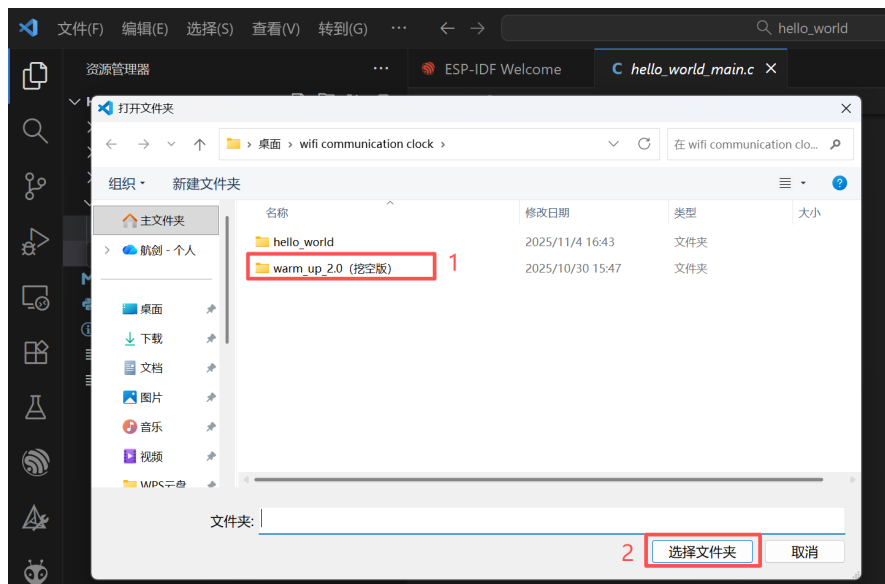
点击“文件”



点击“打开文件夹”



选择 WiFi 数字时钟示例项目文件夹,点击打开



4.3 WIFI 配置

本项目使用局域网连接实现上位机与 ESP32 之间的通信，首先打开自己手机的热点，打开电脑设置，将电脑 WiFi 连接到自己的手机热点。并修改 WiFi 数字时钟项目文件夹中 WiFi 模块的 `wifi.h` 代码：

```
#define EXAMPLE_ESP_WIFI_SSID "B118" 修改为自己的 WiFi 名称  
#define EXAMPLE_ESP_WIFI_PASS "lcme2018" 修改为自己的 WiFi 密码
```

修改完成后，点击“构建、烧录、监视”按钮，看到监视窗口出现下列字符，表示 WiFi 连接已经成功，并且获得了 IP 地址和端口号。



运行 GUI 界面，即可将监视窗口获得的 IP 地址和端口号填到下面的界面中，点击连接服务器后，即可建立上位机和下位机之间的 WiFi 通信。



4.4 补全代码

将示例项目框架中 `/* TODO */` 部分的内容修改为能满足要求的代码。需要补充的代码块包括：

①Wifi 模块中 `tcp_process_command` 函数如何处理接收到的 `cmd` 指令，进而触发计时器计时。

②Counter 模块中 `counter_start` 函数 `cnt` 变量与时钟显示数值之间的运算逻辑关系。

③Wifi 模块中 `tcp_send_data_task` 函数如何将处理得到的时间信息通过 TCP 发送到上位机

推荐按照以上完成顺序，并在每个部分完成后进行测试。

实验二：步进电机-滚珠丝杆实验

1 实验目的

1. 熟悉 ESP32 的外设控制：学习使用 ESP-IDF 的驱动库控制步进电机、处理 GPIO 中断以及读取编码器数据等。
2. 掌握 FreeRTOS 多任务编程：学习在 ESP32 平台上使用 FreeRTOS 创建和管理多个任务，理解任务的生命周期和状态。
3. 理解任务间通信与同步机制：深入掌握 FreeRTOS 的事件标志组（Event Group）和消息队列（Queue）的工作原理，并运用它们实现任务间的精确同步和高效率数据传递。
4. 实现嵌入式网络通信：掌握 ESP32 的 WiFi STA 模式连接，并基于 Socket 编程实现一个简单的 TCP 客户端，能够与上位机进行双向数据通信。
5. 培养实时系统设计思维：通过一个综合项目，理解如何将复杂系统功能分解为独立的并发任务，并设计高效的通信机制来保证系统的实时响应性。

2 任务要求

1. 滑块自动往复运动：在无外部指令且电机启动的状态下，滑块能在两端限位传感器之间自动往复运动，每当限位传感器被触发时，电机停止开始反转。
2. 网络远程控制与实时状态监控：通过 WiFi，系统能向上位机持续发送编码器数据，并接收来自上位机的控制指令（如启动、停止、设定速度等）。
3. （选做）完成思考题所述要求。

3 硬件介绍

1. 步进电机套件

步进电机：42 电机，步距角 1.8° ，200 脉冲/圈，最高转速 600 转/分钟

滑台丝杆螺距：4mm/圈

接近开关：三线制，集电极开路输出，NPN 型

编码器：光电增量式 A、B 两相 600 脉冲，集电极开路输出，NPN 型



图 4

2. 步进电机驱动器

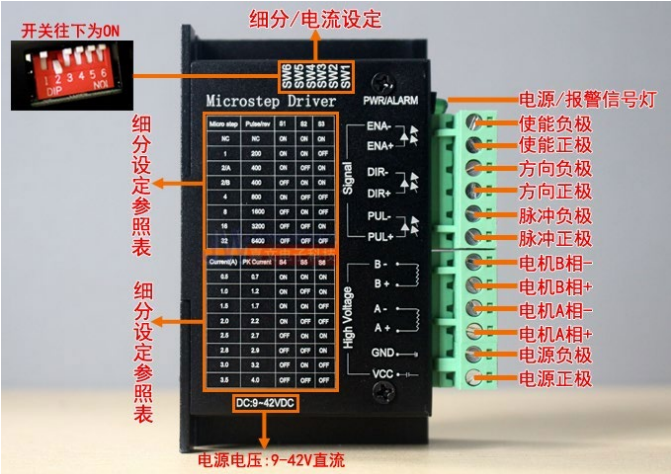


图 5

共阳极接法（低电平有效）

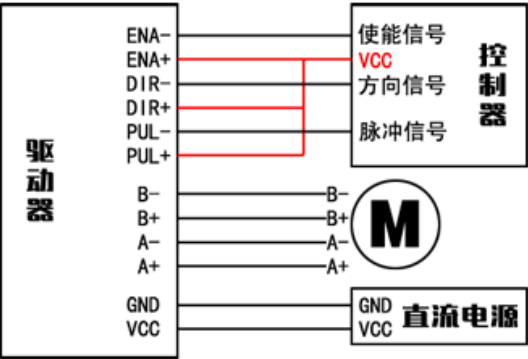


图 6

实验中驱动器控制采用共阳极接法，输出控制信号低电平有效。控制板与驱动器的接线如下面表格所示。驱动器用 DC12V 供电。

3. 直流三线式接近开关

集电极开路输出，NPN 型，工作电压 DC5-30V，在本实验中使用 DC12V 供电。

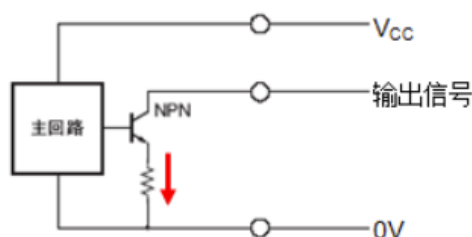


图 7

NPN 集电极开路输出，传感器有效时（即有金属接近），三极管导通，输出信号端电压为 0V。接线如下图：

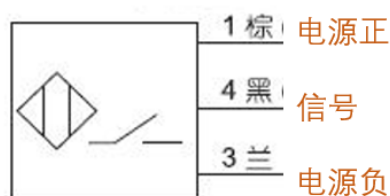


图 8

4. 编码器

增量式 600 脉冲，A、B 两相输出，集电极开路输出，NPN 型。
工作电压：DC 5 - 30V



图 9

4 项目框架介绍

本示例项目基于 ESP-IDF 开发，使用 FreeRTOS 实现了步进电机-滚珠丝杆系统控制的基础框架。系统使用事件组（Event Group）处理传感器紧急中断，利用队列（Queue）实现网络指令的传递。主要模块包括：

- **sensors**: 初始化限位传感器引脚，实现了限位传感器触发的中断服务函数。
- **state_manager**: 管理电机的状态（停止、加速、匀速、减速等），根据当前状态调整电机的速度，实现状态切换逻辑。
- **stepper_motor**: 控制步进电机的使能、方向和速度。

- `wifi`: 管理 WiFi 连接和 TCP 通信, 初始化 WiFi 并连接到指定的 AP。实现全双工的 TCP 服务器。

模块间的依赖关系: `sensors` 和 `wifi` 模块都依赖于 `state_manager`, 在特定事件(如限位触发、上位机下达指令)发生时改变电机运行状态; `sensors` 模块还依赖于 `stepper_motor`, 便于即时控制电机停止; `state_manager` 模块依赖于 `stepper_motor`, 用于根据当前电机状态控制电机。`stepper_motor` 不依赖其他任何自定义组件, 直接实现使能、方向、PWM 信号的调整控制电机, 提供这些操作的接口。

4.1 硬件模块与配置

1. 步进电机驱动模块 `stepper_motor`
 - **控制引脚**: PWM 输出引脚、方向控制引脚、使能引脚
 - **驱动方式**: 采用 LEDC 硬件 PWM 控制, 支持微步进设置
 - **参数配置**:
 - 最大转速: `MAX_RPM`
 - 最小转速: `MIN_RPM`
 - 微步进: `MICRO_STEPS`
2. 限位传感器模块 `sensors`
 - **传感器类型**: GPIO 数字输入传感器, NPN 集电极开路输出
 - **信号引脚**: 左限位(`SENSOR_L_GPIO`)、右限位(`SENSOR_R_GPIO`)
 - **触发方式**: 下降沿触发
 - **中断处理**: 采用 GPIO 中断服务, 通过事件组通知中断处理程序处理
3. 网络通信模块 `wifi`
 - **通信协议**: TCP/IP 协议
 - **工作模式**: STA 模式, 连接指定 Wi-Fi 网络
 - **服务端口**: 3333

4.2 任务设计及优先级分配

系统包含以下主要任务, 按优先级从高到低排列:

1. 状态管理任务 (`state_switch_task()`, 优先级 5)

系统核心控制任务, 负责电机运行状态管理, 实现加速、减速、匀速等状态转换逻辑, 高优先级保证电机控制的实时性。
2. 传感器处理任务 (`sensor_trigger_task`, 优先级 4)

限位传感器紧急处理任务, 通过事件组等待传感器触发事件, 处理电机限位保护和反转逻辑。
3. TCP 服务器任务 (`tcp_server_task`, 优先级 3)

承担了网络连接管理任务, 同时实时监听客户端连接, 接收上位机指令数据, 解析指令并通过队列 `tcp_command_queue` 发送给命令处理任务。

4. TCP 命令处理任务 (tcp_recv_cmd_task, 优先级 2)

从队列 `tcp_command_queue` 中取出指令并执行相应操作, 修改系统状态变量, 影响电机行为。

5. TCP 数据发送任务 (tcp_send_data_task, 优先级 1)

状态数据推送任务, 周期性地向客户端发送电机当前状态信息。

4.3 FreeRTOS 通信机制应用

本项目使用事件组 (Event Group) 进行限位中断服务和中断处理任务之间的通信, 提供快速的中断到任务同步机制, 确保传感器触发事件得到及时处理。使用队列 (Queue) 进行网络指令传递, 队列的发送端是 TCP 服务器任务 `tcp_server_task`, 接收端是 TCP 命令处理任务 `tcp_recv_cmd_task`, 实现任务间数据安全传递, 解耦网络接收和命令执行过程。

系统通过以下全局变量在任务间共享状态:

- `motor_state_t current_status`: 电机当前状态 (停止、加速、匀速、限位停止)
- `int target_spd`: 目标转速值
- `int current_spd`: 当前实际转速值
- `int current_dir`: 电机当前转动方向

并使用互斥量 `mutexHandle` 进行保护。

整个项目的线程架构和通信机制如图 10 所示。

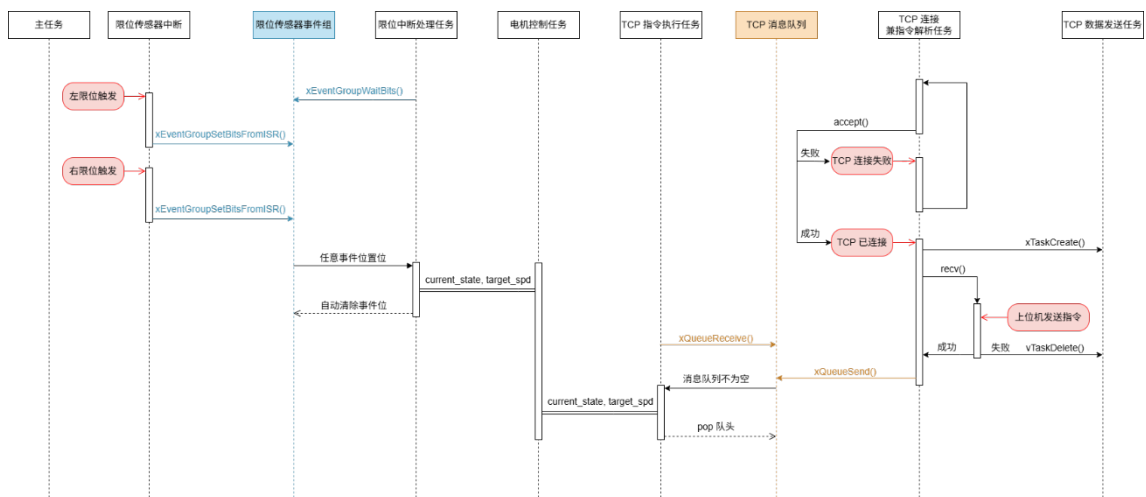


图 10

4.4 GUI

本示例项目使用 PyQt5 开发了一个上位机 GUI 界面, 提供了 ESP32 网络 IP 地址和 TCP 服务端口号设置控件、电机启停按钮、电机速度显示仪表盘、系统状态信息打印框等。GUI 界面截图如图 11 所示。

GUI 的脚本路径为 `./GUI/test_PyQt.py`, 在 VSCode 中打开该脚本, 点击右上角

角的运行 ▶ 按钮或在终端运行命令“python ./GUI/test_PyQt.py”即可打开 GUI 界面，详细步骤见实验一中的 3.2 节。

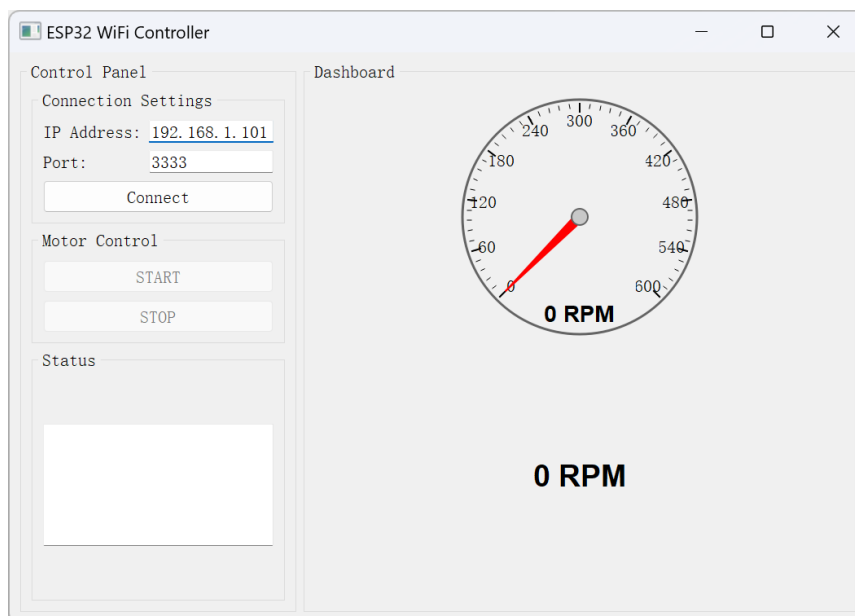
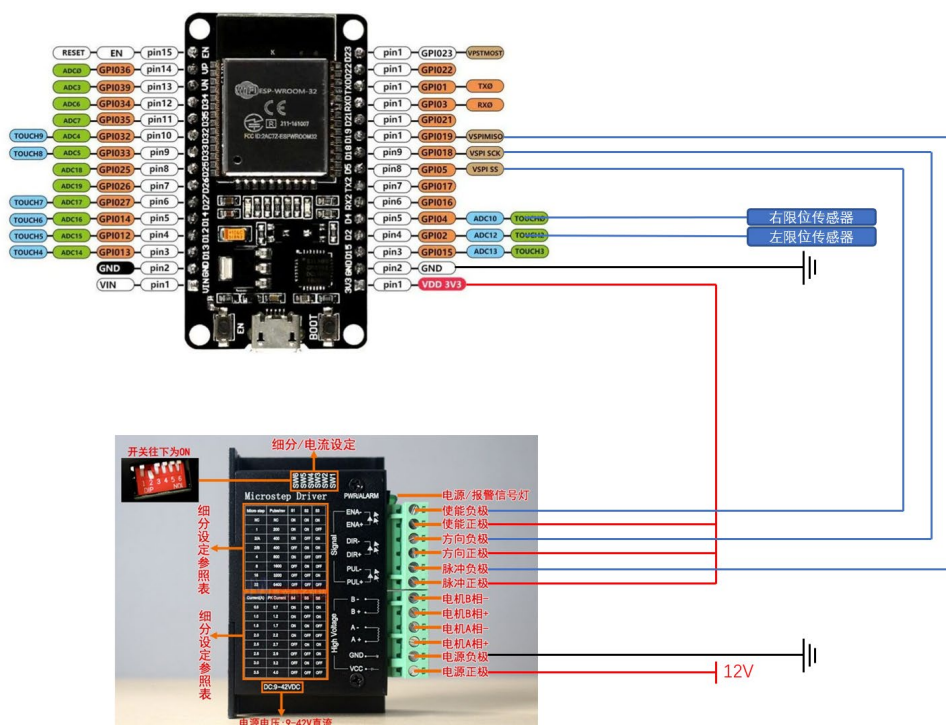


图 11

5 实验步骤指导

5.1 硬件接线

本项目中 ESP32 的 GPIO 口的硬件连接方式如下，建议同学们在硬件连接时以相同的 IO 口连接硬件，就无需更改示例代码中的 IO 口配置（若硬件接线 IO 口不同，则需要去代码中进行更改）。



左限位传感器	GPIO 2
右限位传感器	GPIO 4
驱动器信号正极	共阳，接 V3.3
ENA-	GPIO 5
DIR-	GPIO 18
PUL-	GPIO 19
B+ B- A+ A-	接电机
驱动器供电	12VDC

5.2 WIFI 配置

本项目使用局域网连接实现上位机与 ESP32 之间的通信，首先打开自己手机的热点，打开电脑设置，将电脑 WiFi 连接到自己的手机热点。并修改示例项目中 WiFi 模块的代码：

```
#define EXAMPLE_ESP_WIFI_SSID    "B118"      修改为自己的 WiFi 名称
#define EXAMPLE_ESP_WIFI_PASS    "lcme2018"  修改为自己的 WiFi 密码
```

5.3 补全代码

将示例项目框架中 `/* TODO */` 部分的内容修改为能满足要求的代码。需要补充的代码块包括：

- ① `stepper_motor` 模块中的设置电机速度函数 `int set_motor_speed(int rpm)` ；
- ② `sensor` 模块中限位触发处理任务 `sensor_trigger_task` 当限位触发时的处理逻辑；
- ③ `state_manager` 模块中电机处于加速状态 `STATE_ACCELERATING` 的运行逻辑；
- ④ `wifi` 模块中解析接收到的上位机指令并发送到消息队列供 TCP 指令执行任务读取。

推荐按照以上完成顺序，并在每个部分完成后进行测试，测试成功后请老师或助教验收。

5.4 完成思考题

1. 将编码器接入到 ESP32 开发板，在上位机 GUI 中实时显示编码器反馈的电机速度。（10 分）
2. 实现实时设置电机速度的功能，要求电机速度平滑变化，在实验报告中写出你设置的加速度大小。（6 分）
3. 实现电机在到达两端限位时停止 1s 后再反向重新启动，并说明：①在你的设计中，在限位触发停止的时间内可否由上位机强制让电机提前开始启动。②如何实现相反的设计。（3 分）

FreeRTOS 基础概念与 API

1 实时操作系统与 FreeRTOS

实时操作系统（Real-Time Operating System, RTOS）是一种专为在严格时间限制内响应外部事件而设计的操作系统。它广泛应用于对任务执行时序有确定性要求的嵌入式系统，如医疗设备、工业控制器和汽车电子控制单元等。与 Linux 等通用操作系统不同，专为嵌入式设备设计的 RTOS（如 FreeRTOS）通常更为轻量，由于不支持虚拟内存，其“线程”与“进程”无异，也常称为“任务”。

内核是 RTOS 的核心组件，负责管理系统的硬件资源（如 CPU 时间、内存）并为应用程序提供基础服务。FreeRTOS 是一个开源、小巧、可裁剪的 RTOS 内核，被广泛移植到各种微控制器上。在 ESP32 的开发框架 ESP-IDF 中，FreeRTOS 被作为核心组件集成。

在传统的“裸机”程序（即不带操作系统的程序）中，通常采用单循环架构：一个无限的 `while(1)` 循环依次处理各项事务，如按键扫描、传感器数据读取和显示刷新。这种架构简单，但存在明显缺点：

1. 伪并行与响应延迟：所有任务都在一个循环中顺序执行，任务之间无优先级之分，无法实现真正的并行，系统响应性差。
2. 阻塞操作导致系统停滞：如果某个任务（如等待串口数据）需要长时间等待（阻塞），整个循环都会停下来，其他任务也无法执行。

FreeRTOS 通过多任务并行处理机制解决了上述问题，其核心思想是“分时复用”，如图 12 所示。FreeRTOS 中的核心概念包括：

- **任务**：将应用程序分解为多个独立的、无限循环的执行单元，即“任务”。每个任务都有自己的栈空间和优先级。
- **调度器**：RTOS 的核心，一个始终运行的高优先级程序。它负责决定在任意时刻哪个任务可以占用 CPU 执行。
- **调度机制**：调度器根据任务的优先级和状态（如就绪、阻塞）进行决策。高优先级的就绪任务会立即抢占低优先级任务的 CPU 使用权。此外，当一个任务主动延时或等待信号量、队列等事件时，它会进入“阻塞”状态，调度器会立刻切换到其他就绪的任务去执行，从而高效利用 CPU 时间。

这使得多个任务在宏观上看起来是同时运行的，极大地提高了 CPU 利用率和系统的实时响应能力。

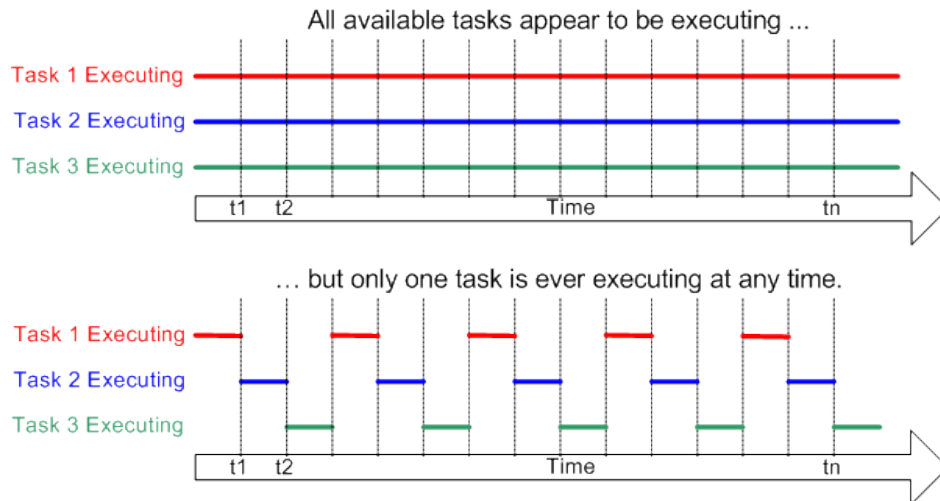


图 12

在 FreeRTOS 中，任务在任何时刻都处于以下四种状态之一，其生命周期和状态转换如下图所示：

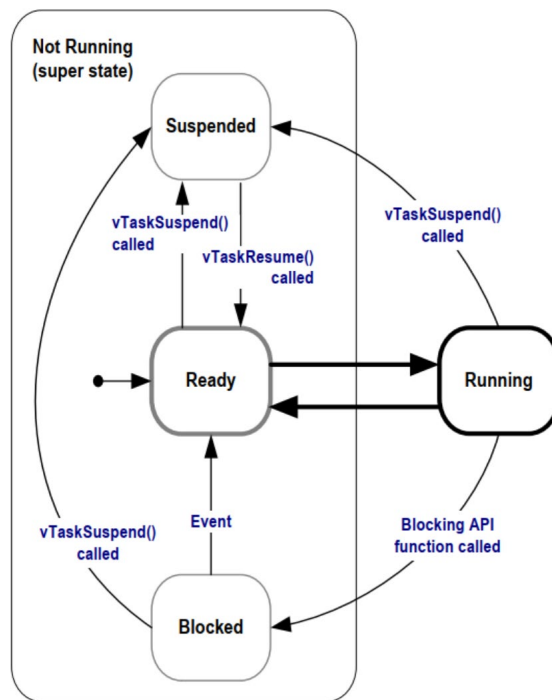


图 13

- 运行：任务正在 CPU 上执行。在单核处理器上，同一时刻只能有一个任务处于运行状态。
- 就绪：任务已经准备就绪可以运行（未被阻塞或挂起），但由于有更高优先级的任务正在运行或当前任务优先级相同但时间片未到，而暂时等待调度器分配 CPU 时间。
- 阻塞：任务正在等待某个事件或延时周期结束。例如，任务调用了 `vTaskDelay(100)` 延时 100 个时钟节拍，或在等待一个信号量时，就会进入

阻塞状态。事件发生或延时结束后，任务会离开阻塞状态，进入就绪状态。

- 挂起：任务被明确地置于休眠状态，调度器不会再调度它，直到被另一个任务通过 `vTaskResume()` API 明确地恢复。挂起操作不会超时。任务可以通过调用 `vTaskSuspend()` 挂起自身。

2 在 ESP-IDF 中创建 FreeRTOS 任务

FreeRTOS 作为核心组件已集成在 ESP-IDF 中，我们可以直接使用其 API 来创建和管理任务。

在 ESP-IDF 中，我们使用 `xTaskCreate()` 函数来动态地创建一个任务。

```
BaseType_t xTaskCreate(TaskFunction_t pvTaskCode,
                      const char * const pcName,
                      const uint32_t usStackDepth,
                      void *pvParameters,
                      UBaseType_t uxPriority,
                      TaskHandle_t *pvCreatedTask);
```

- `pvTaskCode`: 指向任务实现函数的指针。此函数通常是一个无限循环。
- `pcName`: 任务的描述性名称，主要用于调试。
- `usStackDepth`: 任务栈的大小，单位是字（在 ESP32 上为 4 字节）。需要根据任务内局部变量、函数调用深度来合理分配。
- `pvParameters`: 传递给任务函数的参数。
- `uxPriority`: 任务的优先级。数字越大，优先级越高。优先级范围由 `configMAX_PRIORITIES` 决定。
- `pvCreatedTask`: 用于传回任务句柄 (Handle)，可通过此句柄来引用该任务（如删除、恢复任务）。如果不需要，可设为 `NULL`。

函数返回 `pdPASS` 表示任务创建成功，返回 `pdFAIL`（通常是 `errCOULD_NOT_ALLOCATE_REQUIRED_MEMORY`）表示创建失败（例如内存不足）。

创建任务示例代码

```
#include <stdio.h>
#include "esp_log.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
static const char *TAG = "main";
// 任务函数
void myTask(void *pvParameters)
{
    for (;;)
    {
```

```
        vTaskDelay(1000 / portTICK_PERIOD_MS);
        ESP_LOGI(TAG, "myTask");
    }
}

void app_main(void)
{
    // 创建一个 FreeRTOS 任务
    // 参数说明:
    // 1. 任务入口函数: myTask
    // 2. 任务名称: "myTask", 用于调试时标识任务
    // 3. 任务堆栈大小: 2048 字节 (适当分配以避免栈溢出)
    // 4. 任务参数: NULL (未传递参数)
    // 5. 任务优先级: 1 (优先级较低, 空闲任务优先级为 0)
    // 6. 任务句柄: NULL (不需要保存任务句柄)
    xTaskCreate(myTask, "myTask", 2048, NULL, 1, NULL);
}
```

传递参数示例代码

```
// 任务函数 1:接收整型参数
void Task_1(void *pvParameters)
{
    int *pInt = (int *)pvParameters;
    ESP_LOGI(TAG, "取得整型参数为: %d", *pInt);
    vTaskDelete(NULL);
}

// 任务函数 2:接收数组参数
void Task_2(void *pvParameters)
{
    int *pArray = (int *)pvParameters;
    ESP_LOGI(TAG, "取得数组参数为: %d %d %d", *pArray, *(pArray + 1), *(pArray + 2));
    vTaskDelete(NULL);
}

int Parameters_1 = 1;
int Parameters_2[3] = {1, 2, 3};
void app_main(void)
{
    // 传递整形参数
    xTaskCreate(Task_1, "Task_1", 2048, (void *)&Parameters_1, 1, NULL);
    // 传递数组参数
    xTaskCreate(Task_2, "Task_2", 2048, (void *)&Parameters_2, 1, NULL);
}
```

FreeRTOS 任务挂起

```

TaskHandle_t taskHandle_1 = NULL;
// 任务 1: 定时打印日志
void Task_1(void *pvParameters)
{
    while (1)
    {
        ESP_LOGI(TAG, "任务 1 正在运行...");
        vTaskDelay(pdMS_TO_TICKS(1000)); // 延迟 1 秒
    }
}
// 任务 2: 控制任务 1 的挂起与恢复
void Task_2(void *pvParameters)
{
    while (1)
    {
        ESP_LOGI(TAG, "挂起任务 1...");
        vTaskSuspend(taskHandle_1); // 挂起任务 1
        vTaskDelay(pdMS_TO_TICKS(5000)); // 延迟 5 秒
        ESP_LOGI(TAG, "恢复任务 1...");
        vTaskResume(taskHandle_1); // 恢复任务 1
        vTaskDelay(pdMS_TO_TICKS(5000)); // 再延迟 5 秒
    }
}
void app_main(void)
{
    xTaskCreate(Task_1, "Task_1", 2048, NULL, 5, &taskHandle_1);
    xTaskCreate(Task_2, "Task_2", 2048, NULL, 5, NULL);
}

```

3 在 ESP-IDF 中实现任务间通信

在多任务系统中，各个任务虽然是独立运行的，但一般需要协同工作以完成共同的系统功能，这就需要在任务之间传递数据或同步彼此的执行状态。FreeRTOS 提供了多种机制来实现任务间通信（Inter-Task Communication, ITC），其中最常用的是队列、信号量、互斥锁和事件组。

3.1 队列

队列用于在任务之间传递消息，遵循先进先出的原则，但也支持后进先出的插入方式。发送和接收任务可以处于不同的优先级，并且支持超时等待机制。

核心 API:

```
QueueHandle_t xQueueCreate(UBaseType_t uxQueueLength, UBaseType_t uxItemSize)
```


- 功能：创建一个队列。
- 参数：`uxQueueLength`-队列可容纳的最大消息数量；`uxItemSize`-队列中每个消息的大小（字节）。
- 返回值：成功则返回队列句柄，失败（内存不足）返回 NULL。

```
BaseType_t xQueueSend(QueueHandle_t xQueue
                      const void * pvItemToQueue,
                      TickType_t xTicksToWait)
```

- 功能：向队列尾部发送一条消息。
- 参数：`xQueue`-队列句柄；`pvItemToQueue`-指向要发送数据的指针；`xTicksToWait`-如果队列已满，任务阻塞等待的最大时间。设为 0 表示不等待立即返回，设为 `portMAX_DELAY` 表示无限期等待。

```
BaseType_t xQueueReceive(QueueHandle_t xQueue,
                        void *pvBuffer,
                        TickType_t xTicksToWait)
```

- 功能：从队列头部接收一条消息。
- 参数：`xQueue`-队列句柄；`pvBuffer`-指向接收数据缓冲区的指针；`xTicksToWait`-如果队列为空，任务阻塞等待的最大时间。

```
QueueHandle_t xQueue;
xQueue = xQueueCreate(10, sizeof(int)); // 创建一个可以存储 10 个整数的队列
if (xQueue == NULL) {
    // 队列创建失败，处理错误
}

int data = 42;
if (xQueueSend(xQueue, &data, 0) != pdPASS) {
    // 数据发送失败，处理错误
}

int receivedData;
if (xQueueReceive(xQueue, &receivedData, portMAX_DELAY) == pdPASS) {
    // 成功接收数据，进行处理
}
```

队列传参示例代码

```
void Task_1(void *pvParameters)
{
    // 取得队列句柄
    QueueHandle_t xQueue = (QueueHandle_t)pvParameters;
    int i = 0;

    for (;;)
    {
```

```
{
    // 发送数据到队列
    if (xQueueSend(xQueue, &i, 0) != pdPASS) {
        ESP_LOGI(TAG, "数据发送失败");
    }
    else
    {
        ESP_LOGI(TAG, "数据发送成功");
        i++;
    }

    if(i == 10)
    {
        i = 0;
    }
    vTaskDelay(1000 / portTICK_PERIOD_MS);
}
vTaskDelete(NULL);
}

void Task_2(void *pvParameters)
{
    // 取得队列句柄
    QueueHandle_t xQueue = (QueueHandle_t)pvParameters;
    for (;;)
    {
        int receivedData;
        if (xQueueReceive(xQueue, &receivedData, 0) != pdPASS)
        {
            ESP_LOGI(TAG, "数据接收失败");
        }
        else
        {
            ESP_LOGI(TAG, "数据接收成功, 数据为: %d", receivedData);
        }
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
    vTaskDelete(NULL);
}

void app_main(void)
{
    TaskHandle_t taskHandle_1 = NULL;
    TaskHandle_t taskHandle_2 = NULL;
    QueueHandle_t xQueue;
    // 创建队列
```

```

xQueue = xQueueCreate(10, sizeof(int));

if (xQueue != NULL)
{
    ESP_LOGI(TAG, "队列创建成功");
    // 发送数据任务
    xTaskCreate(Task_1, "Task_1", 1024 * 4, (void *)xQueue, 12, &taskHandle_1);
    // 接收数据任务
    xTaskCreate(Task_2, "Task_1", 1024 * 4, (void *)xQueue, 12, &taskHandle_2);
}
else
{
    ESP_LOGI(TAG, "队列创建失败");
}
}

```

3.2 信号量与互斥锁

信号量是一种用于同步和互斥访问的机制。它可以被看作一个计数器，用于管理对多个共享资源的访问。FreeRTOS 中主要使用二进制信号量和计数信号量。

1. 二进制信号量

二进制信号量只有 0 和 1 两种状态，常用于同步（如中断服务例程中通知任务）或互斥（保护共享资源）。

`SemaphoreHandle_t xSemaphoreCreateBinary(void)`: 创建一个二进制信号量，初始状态为 0（不可用）。

`xSemaphoreGive(SemaphoreHandle_t xSemaphore)`: 释放（增加）信号量。

`xSemaphoreTake(SemaphoreHandle_t xSemaphore, TickType_t xBlockTime)`: 获取（减少）信号量。

```

// 二进制信号量
SemaphoreHandle_t semaphoreHandle;
// 公共变量
int shareVariable = 0;

void task1(void *pvParameters)
{
    for (;;)
    {
        // 获取信号量,信号量变为 0
        xSemaphoreTake(semaphoreHandle, portMAX_DELAY);
        for (int i = 0; i < 10; i++)
        {

```

```

        shareVariable++;
        ESP_LOGI(TAG, "task1 shareVariable:%d", shareVariable);
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
    // 释放信号量,信号量变为1
    xSemaphoreGive(semaphoreHandle);
    vTaskDelay(pdMS_TO_TICKS(1000));
}
}
void task2(void *pvParameters)
{
    for (;;)
    {
        // 获取信号量
        xSemaphoreTake(semaphoreHandle, portMAX_DELAY);
        for (int i = 0; i < 10; i++)
        {
            shareVariable++;
            ESP_LOGI(TAG, "task2 shareVariable:%d", shareVariable);
            vTaskDelay(pdMS_TO_TICKS(1000));
        }
        // 释放信号量,信号量变为1
        xSemaphoreGive(semaphoreHandle);
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

void app_main(void)
{
    semaphoreHandle = xSemaphoreCreateBinary();
    xSemaphoreGive(semaphoreHandle);
    xTaskCreate(task1, "task1", 1024 * 2, NULL, 10, NULL);
    xTaskCreate(task2, "task2", 1024 * 2, NULL, 10, NULL);
}

```

2. 互斥锁

互斥锁是一种特殊的二进制信号量，它引入了优先级继承机制，用于解决优先级反转问题，专门用于保护共享资源（临界区）。

`SemaphoreHandle_t xSemaphoreCreateMutex(void)`: 创建一个互斥锁。

```

SemaphoreHandle_t mutexHandle;
void task1(void *pvParameters)
{
    ESP_LOGI(TAG, "task1 启动!");
}

```

```
while (1)
{
    if (xSemaphoreTake(mutexHandle, portMAX_DELAY) == pdTRUE)
    {
        ESP_LOGI(TAG, "task1 获取到互斥量!");
        for (int i = 0; i < 10; i++)
        {
            ESP_LOGI(TAG, "task1 执行中...%d", i);
            vTaskDelay(pdMS_TO_TICKS(1000));
        }
        xSemaphoreGive(mutexHandle);
        ESP_LOGI(TAG, "task1 释放互斥量!");
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
    else
    {
        ESP_LOGI(TAG, "task1 获取互斥量失败!");
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

void task2(void *pvParameters)
{
    ESP_LOGI(TAG, "task2 启动!");
    vTaskDelay(pdMS_TO_TICKS(1000));
    while (1)
    {
    }
}

void task3(void *pvParameters)
{
    ESP_LOGI(TAG, "task3 启动!");
    vTaskDelay(pdMS_TO_TICKS(1000));
    while (1)
    {
        if (xSemaphoreTake(mutexHandle, 1000) == pdPASS)
        {
            ESP_LOGI(TAG, "task3 获取到互斥量!");
            for (int i = 0; i < 10; i++)
            {
                ESP_LOGI(TAG, "task3 执行中...%d", i);
                vTaskDelay(pdMS_TO_TICKS(1000));
            }
        }
    }
}
```

```

        xSemaphoreGive(mutexHandle);
        ESP_LOGI(TAG, "task3 释放互斥量!");
        vTaskDelay(pdMS_TO_TICKS(5000));
    }
    else
    {
        ESP_LOGI(TAG, "task3 未获取到互斥量!");
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
}

void app_main(void)
{
    mutexHandle = xSemaphoreCreateMutex();
    xTaskCreate(task1, "task1", 1024 * 2, NULL, 1, NULL);
    xTaskCreate(task2, "task2", 1024 * 2, NULL, 2, NULL);
    xTaskCreate(task3, "task3", 1024 * 2, NULL, 3, NULL);
}

```

3.3 事件组

事件组 (Event Group) 是一种轻量级的机制, 用于任务间的事件通知和同步。一个事件组本质上是一个位掩码 (bit mask), 其中每一位 (bit) 代表一个独立的事件标志 (Event Flag)。任务可以设置 (置位) 事件组中的位来通知其他任务发生了某个事件, 也可以等待 (清除) 事件组中的一个或多个位被置位。

事件组特别适用于以下场景:

1. 等待多个事件中的任意一个发生: 例如, 一个任务需要等待按键按下 或者 定时器超时。
2. 等待多个事件全部发生: 例如, 一个任务需要等待传感器 A 数据准备好 并且 传感器 B 数据准备好 并且 网络连接建立成功。
3. 广播事件给多个任务: 一个任务设置事件位, 多个任务可以同时等待该位。

核心 API:

```
EventGroupHandle_t xEventGroupCreate(void)
```

- 功能: 创建一个新的事件组。
- 返回值: 成功则返回事件组句柄, 失败 (内存不足) 返回 NULL。

```
EventBits_t xEventGroupSetBits(EventGroupHandle_t xEventGroup,
                                const EventBits_t uxBitsToSet)
```

- 功能: 设置 (置位) 事件组中的一个或多个位。
- 参数: xEventGroup-事件组句柄; uxBitsToSet-一个位掩码, 指定要置位的

位。例如，要置位 bit 0 和 bit 2，则 `uxBitsToSet = (1 << 0) | (1 << 2)`。

- 返回值：事件组在设置操作之后的值（位掩码）。通常用于调试。

```
EventBits_t xEventGroupClearBits(EventGroupHandle_t xEventGroup,
                                const EventBits_t uxBitsToClear)
```

- 功能：清除（清零）事件组中的一个或多个位。
- 参数：xEventGroup-事件组句柄；uxBitsToClear-一个位掩码，指定要清除的位。
- 返回值：事件组在清除操作之前的值（位掩码）。

```
EventBits_t xEventGroupWaitBits(const EventGroupHandle_t xEventGroup,
                                const EventBits_t uxBitsToWaitFor,
                                const BaseType_t xClearOnExit,
                                const BaseType_t xWaitForAllBits,
                                TickType_t xTicksToWait)
```

- 功能：等待事件组中的一个或多个位被置位。这是任务同步的关键函数。
- 参数：
 - ◆ xEventGroup：事件组句柄。
 - ◆ uxBitsToWaitFor：一个位掩码，指定要等待哪些位被置位。
 - ◆ xClearOnExit：如果设置为 `pdTRUE`，则在函数退出（成功返回）时，自动清除 `uxBitsToWaitFor` 指定的位。
 - ◆ xWaitForAllBits：指定等待条件。
 - ◆ `pdTRUE`：等待 `uxBitsToWaitFor` 指定的所有位都被置位（逻辑与）。
 - ◆ `pdFALSE`：等待 `uxBitsToWaitFor` 指定的任意一位被置位（逻辑或）。
 - ◆ xTicksToWait：等待的最大时间（时钟节拍数）。设为 0 表示不等待立即返回，设为 `portMAX_DELAY` 表示无限期等待。
- 返回值：如果成功等到指定条件，返回事件组的值（位掩码），此时该值满足等待条件。如果超时，则返回事件组当前的值（可能部分满足条件）。如果 `xClearOnExit` 为 `pdTRUE`，返回的值是清除指定位之前的值。

```
EventBits_t uxBits;
uxBits = xEventGroupWaitBits(
    xEventGroup,      // 事件组句柄
    0x03,             // 等待第 0 位和第 1 位
    pdTRUE,           // 退出等待时清除事件标志
    pdFALSE,          // 等待任意一个事件
    portMAX_DELAY     // 无限等待
);

if (uxBits & 0x01) {
    // 第 0 位事件发生
}
```



```
if (uxBits & 0x02) {  
    // 第 1 位事件发生  
}
```

事件组等待示例代码

task1 等待 task2 设置事件位，然后执行程序：

```
EventGroupHandle_t xCreatedEventGroup;  
  
#define BIT_0 (1 << 0)  
#define BIT_4 (1 << 4)  
  
void task1(void *pvParameters)  
{  
    ESP_LOGI(TAG, "-----");  
    ESP_LOGI(TAG, "task1 启动!");  
  
    while (1)  
    {  
        EventBits_t uxBits;  
        uxBits = xEventGroupWaitBits(  
            xCreatedEventGroup,  
            BIT_0 | BIT_4,  
            pdTRUE,  
            pdFALSE,  
            portMAX_DELAY);  
        if ((uxBits & (BIT_0 | BIT_4)) == (BIT_0 | BIT_4))  
        {  
            ESP_LOGI(TAG, "BIT_0 和 BIT_4 都被设置了");  
        }  
        else  
        {  
            ESP_LOGI(TAG, "BIT_0 和 BIT_4 有一个被设置了");  
        }  
    }  
}  
  
void task2(void *pvParameters)  
{  
    ESP_LOGI(TAG, "task2 启动!");  
    vTaskDelay(pdMS_TO_TICKS(1000));  
    while (1)  
    {  
        xEventGroupSetBits(xCreatedEventGroup, BIT_0);  
        ESP_LOGI(TAG, "BIT_0 被设置");  
        vTaskDelay(pdMS_TO_TICKS(3000));  
    }  
}
```

```

        xEventGroupSetBits(xCreatedEventGroup, BIT_4);
        ESP_LOGI(TAG, "BIT_4 被设置");
        vTaskDelay(pdMS_TO_TICKS(3000));
    }
}

void app_main(void)
{
    // 创建事件组
    xCreatedEventGroup = xEventGroupCreate();

    if (xCreatedEventGroup == NULL)
    {
        ESP_LOGE(TAG, "创建事件组失败");
    }
    else
    {
        xTaskCreate(task1, "task1", 1024 * 2, NULL, 1, NULL);
        xTaskCreate(task2, "task2", 1024 * 2, NULL, 1, NULL);
    }
}

```

事件组同步示例代码

每个任务在启动后，等待一段时间，然后调用 `xEventGroupSync` 函数进行事件同步，等待所有任务的事件位都被设置。

```

/* Bits used by the three tasks. */
#define TASK_0_BIT (1 << 0)
#define TASK_1_BIT (1 << 1)
#define TASK_2_BIT (1 << 2)
#define ALL_SYNC_BITS (TASK_0_BIT | TASK_1_BIT | TASK_2_BIT)

static const char *TAG = "main";
EventGroupHandle_t xEventBits;
void task0(void *pvParameters)
{
    ESP_LOGI(TAG, "-----");
    ESP_LOGI(TAG, "task0 启动!");

    while (1)
    {
        vTaskDelay(pdMS_TO_TICKS(3000));
        ESP_LOGI(TAG, "task0: 任务同步开始");
        // 事件同步
        xEventGroupSync(

```

```
        xEventBits,    /* The event group being tested. */
        TASK_0_BIT,    /* The bits within the event group to wait for. */
        ALL_SYNC_BITS, /* The bits within the event group to wait for. */
        portMAX_DELAY); /* Wait a maximum of 100ms for either bit to be set. */

    ESP_LOGI(TAG, "task0: 任务同步完成");
    vTaskDelay(pdMS_TO_TICKS(3000));
}
}

void task1(void *pvParameters)
{
    ESP_LOGI(TAG, "-----");
    ESP_LOGI(TAG, "task1 启动!");

    while (1)
    {
        vTaskDelay(pdMS_TO_TICKS(4000));
        ESP_LOGI(TAG, "task1: 任务同步开始");

        // 事件同步
        xEventGroupSync(
            xEventBits,    /* The event group being tested. */
            TASK_1_BIT,    /* The bits within the event group to wait for. */
            ALL_SYNC_BITS, /* The bits within the event group to wait for. */
            portMAX_DELAY); /* Wait a maximum of 100ms for either bit to be set. */

        ESP_LOGI(TAG, "task1: 任务同步完成");
        vTaskDelay(pdMS_TO_TICKS(3000));
    }
}

void task2(void *pvParameters)
{
    ESP_LOGI(TAG, "-----");
    ESP_LOGI(TAG, "task2 启动!");

    while (1)
    {
        vTaskDelay(pdMS_TO_TICKS(5000));
        ESP_LOGI(TAG, "task2: 任务同步开始");
        // 事件同步
        xEventGroupSync(
            xEventBits,    /* The event group being tested. */
            TASK_2_BIT,    /* The bits within the event group to wait for. */
            ALL_SYNC_BITS, /* The bits within the event group to wait for. */
```

```
portMAX_DELAY); /* Wait a maximum of 100ms for either bit to be set. */

    ESP_LOGI(TAG, "task2: 任务同步完成");
    vTaskDelay(pdMS_TO_TICKS(3000));
}
}

void app_main(void)
{
    // 创建事件组
    xEventBits = xEventGroupCreate();

    if (xEventBits == NULL)
    {
        ESP_LOGE(TAG, "创建事件组失败");
    }
    else
    {
        xTaskCreate(task0, "task0", 1024 * 2, NULL, 1, NULL);
        xTaskCreate(task1, "task1", 1024 * 2, NULL, 1, NULL);
        xTaskCreate(task2, "task2", 1024 * 2, NULL, 1, NULL);
    }
}
```

网络相关概念与 API

1 网络通信基本概念

本实验所涉及网络通信相关的基本概念包括：

1.1 TCP/IP 网络模型

TCP/IP 网络模型由 OSI 模型演化而来，将 OSI 模型由七层简化为四层，分别为网络接口层、网络层、传输层和应用层（图 14）。

应用层（Application layer）是网络体系结构中的最高层，定义了应用进程间的通信和交互规则，实现具体的网络应用。应用层协议种类繁多，如 HTTP、FTP、Telnet、DNS、SMTP 等。在应用层交互的数据单元称为报文（message）。

传输层（Transport layer）负责向两台主机中进程之间的通信提供数据传输服务，为不同的网络应用提供通用的接口。传输层使用的主要协议是传输控制协议 TCP（Transmission Control Protocol）和用户数据报协议 UDP（User Datagram Protocol）。TCP 提供面向连接的、可靠的数据传输服务，UDP 提供无连接的、尽最大努力的数据传输服务。

网络层（Net layer）负责为不同主机提供通信服务。在 TCP/IP 体系中，网络层使用 IP 协议，因此网络层的数据传输单元称为 IP 数据报。

网络接口层也称数据链路层（Link layer），负责将 IP 数据报封装成帧，确保数据在相邻节点之间的可靠传输。

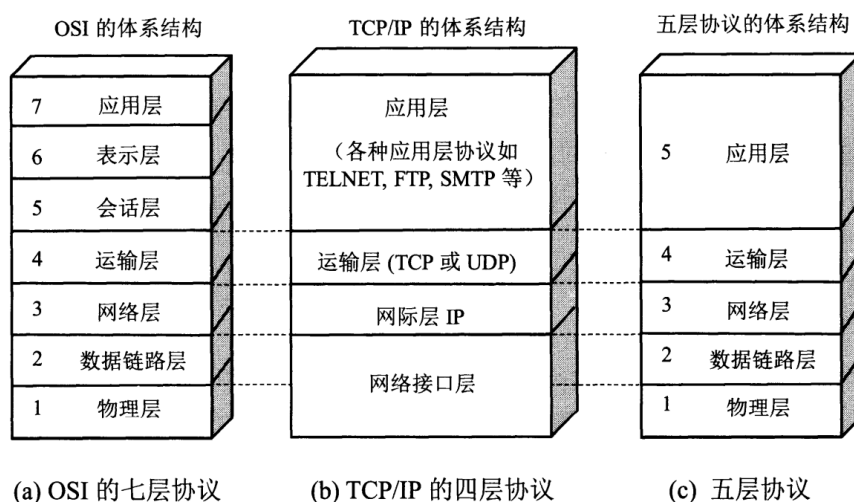


图 14 计算机网络体系结构¹

IP 是 TCP/IP 中最核心的协议，工作在网络层，提供不可靠、无连接的服务。所有的 TCP、UDP、ICMP 等协议均以 IP 数据报的格式传输，IP 协议不保证数据

¹ 谢希仁，计算机网络（第七版）

报一定可以送达目的，也不保证数据报的先后次序。

TCP 是一种**面向连接的、可靠的、基于字节流的传输层通信协议**。它位于网络协议栈的传输层，为应用层提供了一个可靠的数据传输服务。与 UDP（User Datagram Protocol，用户数据报协议）不同，TCP 协议通过一系列的机制确保数据在传输过程中不丢失、不重复且按序到达，因此适用于对数据准确性要求较高的应用场景。

在本实验中，我们基于 ESP32 平台实现了一个完整的 TCP/IP 网络通信应用，TCP/IP 网络模型中的多个层次的使用具体如下：

1. **网络接口层**：使用 WiFi 技术（IEEE 802.11）作为物理和数据链路层的基础，通过 ESP32 的 WiFi 驱动实现无线通信。

2. **网络层**：使用 IP 协议，通过 DHCP 动态获取 IP 地址。当 WiFi 连接成功后，系统会触发 IP_EVENT_STA_GOT_IP 事件，表明已经获取到 IP 地址，此时可以启动 TCP 服务器。

3. **传输层**：使用 TCP 协议，在 3333 端口上建立 TCP 服务器。TCP 提供可靠的连接，确保指令和状态数据准确无误地传输。代码中通过创建 TCP 套接字，绑定地址和端口，并监听连接请求。当客户端连接后，使用 `accept` 函数接受连接，然后通过 `recv` 和 `send` 函数进行数据收发。

4. **应用层**：基于 TCP 的简单文本协议，可以认为我们自定义了一个简单的应用层协议，基于文本格式进行上位机与 ESP32 间的指令和数据交换。

1.2 WiFi 工作模式

AP（Access Point）和 STA（Station）是 WiFi 模块两种最常见的工作模式，对应于不同的网络角色和功能。ESP32 芯片集成了 WiFi 模块，提供 3 种 WiFi 工作模式，分别是 STA、AP、STA+AP 混合模式。

1. STA 模式

STA 模式将设备定义为无线网络的客户端，通过连接到 AP 提供的网络进行通信。在本实验中，ESP32 被设置为 STA 模式，作为客户端连接到现有的 WiFi 网络（可以是个人手机热点或者实验室 WiFi），通过 DHCP 从路由器自动动态获取 IP 地址。

2. AP 模式

AP 是无线网络的中心节点，负责创建并管理一个独立的无线网络。所有连接到该 AP 的设备（STA）之间的通信必须通过 AP 中转，家庭路由器是 AP 模式的典型代表。本实验中，个人手机开启热点作为 AP，上位机 PC 与 ESP32 同时接入到手机热点的局域网内进行通信。

3. STA+AP 模式

当 ESP32 工作于 STA+AP 模式时，首先，作为接入点，手机等其他终端可以连接到该 ESP32；同时该 ESP32 还可以作为端点连接到其他接入点，如路

由器。

1.3 套接字 (Socket)

套接字是网络编程的接口，是应用层与传输层之间的桥梁。在本实验中，通过创建一个流式套接字实现了 TCP 网络通信。

服务器启动时首先将套接字与本地所有网络接口的特定端口 (3333) 进行绑定，使其能够监听来自任意网络路径的连接请求。随后套接字进入监听状态，维护一个待处理连接队列以应对可能的并发访问。

当客户端连接请求到达时，套接字接口通过 `recv()` 函数建立独立的通信通道，创建专用于该连接的新套接字实例，而原监听套接字继续维持监听。建立连接后，套接字为双向数据传输提供可靠保障，确保应用层指令和状态信息能够有序且完整地传输，同时自动处理底层可能发生的丢包、重传和排序等网络异常情况。

套接字作为共享资源被不同任务协作使用，接收任务持续监控指令数据到达，发送任务定期通过套接字上报设备状态，从而实现上位机远程控制与实时监控的双功能。

2 WiFi 连接相关 API

2.1 WiFi 初始化与启动

1. `esp_wifi_init(&cfg);`

功能：初始化 WiFi 驱动

参数：`wifi_init_config_t` 结构体，包含 WiFi 初始化配置，在本实验中采用 esp 库提供的默认配置

使用：必须首先调用，为 WiFi 功能分配资源

2. `esp_wifi_set_mode(WIFI_MODE_STA);`

功能：设置 WiFi 工作模式

参数：模式枚举值 (STA 站模式/AP 接入点模式)，本实验中 ESP 作为 STA 连接到作为 AP 个人热点。

使用：设置设备为 WiFi 客户端连接路由器

3. `esp_wifi_set_config(WIFI_IF_STA, &wifi_config);`

功能：配置 WiFi 连接参数

参数：接口类型和包含 SSID/密码的配置结构体

使用：设置要连接的热点名称和密码

4. `esp_netif_create_default_wifi_sta();`

功能：创建默认的 STA(Station)模式网络接口

使用：配置设备作为 WiFi 客户端

5. `esp_wifi_start();`

功能：启动 WiFi 功能

使用：在配置完成后启动 WiFi

6. `esp_wifi_set_mode(WIFI_MODE_STA);`

功能：设置 WiFi 工作模式

参数：wifi_mode_t 模式（如 WIFI_MODE_STA）

返回值：esp_err_t 类型

7. `esp_wifi_connect();`

功能：连接到指定的 AP

返回值：esp_err_t 类型

2.2 WiFi 事件处理

1. `esp_event_loop_create_default();`

功能：创建默认事件循环

使用：处理 WiFi 连接状态变化等系统事件

2. `esp_event_handler_instance_register();`

功能：注册事件处理回调函数

参数：事件基类型、事件 ID、回调函数、上下文、句柄

使用：监听 WiFi 连接状态和 IP 获取事件并执行相应处理

3 TCP 通信相关 API

注：在 ESP-IDF 中通过 LwIP（轻量级 IP 协议栈）提供一系列标准的 POSIX Socket API，因此必须在 wifi 模块的 CMakeLists 里添加依赖 `lwip`。

1. `socket(AF_INET, SOCK_STREAM, 0);`

功能：创建 TCP 套接字

参数：协议族（AF_INET 表示 IPv4）、套接字类型（SOCK_STREAM 表示 TCP 字节流）、协议（0 表示根据前两个参数自动选择协议）

返回值：套接字描述符（负数为失败）

2. `setsockopt(sock_listen, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));`

功能：设置套接字选项

返回值：0 成功，-1 失败

3. `bind(sock_listen, (struct sockaddr *)&server_addr, sizeof(server_addr));`

功能：绑定套接字到本地地址和端口

参数：套接字描述符、服务器地址结构体、结构体大小

使用：指定服务器监听的 IP（使用 `INADDR_ANY` 绑定到所有可用的网络接口）和端口（指定 3333）

4. `listen(sock_listen, 5);`

功能：开始监听连接请求

参数：套接字描述符、最大等待连接数

使用：将套接字设为监听状态

5. `accept(sock_listen, (struct sockaddr *)&client_addr, &sin_size);`

功能：接受客户端连接（阻塞函数）

参数：监听套接字、客户端地址结构体、地址长度指针

返回值：新的套接字描述符用于数据传输

6. `recv(sock_client, recv_buf, sizeof(recv_buf) - 1, 0);`

功能：接收客户端数据

参数：客户端套接字、接收缓冲区、缓冲区大小、标志

返回值：实际接收的字节数

7. `send(sock_client, data_msg, strlen(data_msg), 0);`

功能：向客户端发送数据

参数：客户端套接字、发送数据缓冲区、数据长度、标志

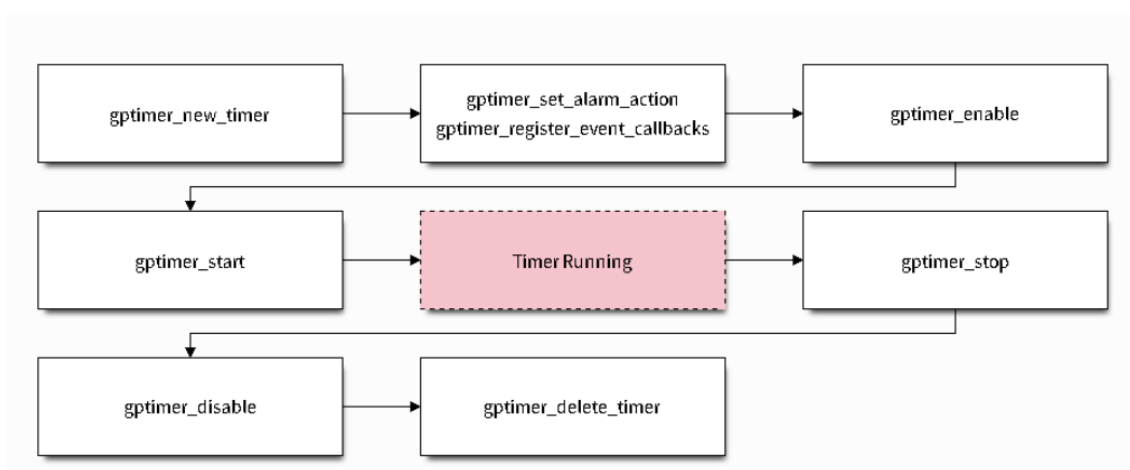
返回值：实际发送的字节数

通用定时器介绍与 API

1 ESP32 通用定时器介绍

1.1 概述

通用定时器是 ESP32 [定时器组外设] 的专用驱动程序。该定时器可以选择不同的时钟源和分频系数，能满足纳秒级的分辨率要求。此外，它还具有灵活的超时报警功能，并允许在报警时刻自动更新计数值，从而实现非常精准的定时周期。



芯片内置 4 个 64-bit 通用定时器（GPTimer, General Purpose Timer），具有 16-bit 分频器和 64-bit 可自动重载的向上/向下计时器。

定时器特性：

- 16-bit 时钟分频器，分频系数为 2 至 65536
- 64-bit 计时器
- 计时器方向可配置：递增或递减
- 软件控制计数暂停和继续
- 定时器超时自动重载
- 软件控制的即时重载
- 电平触发中断和边沿触发中断

用途为：适用于需要高精度定时的场景（如周期性任务、时间测量等），本 Wifi 数字时钟的核心定时器即为此类型。

1.2 通用定时器时钟源

在 ESP32 中通用定时器支持多种时钟源，可通过 `gptimer_clk_src_t` 枚举类型进行配置。根据 ESP-IDF 官方文档及硬件特性，通用定时器可用的时钟源主要包括以下几种：

1. `GPTIMER_CLK_SRC_DEFAULT`（默认时钟源）

对应时钟：通常为 APB_CLK（外设总线时钟）。

频率：默认配置下为 80MHz（当 CPU 主频为 160MHz 时，APB_CLK 为 80MHz；若 CPU 主频为 240MHz，APB_CLK 可配置为 80MHz 或 120MHz，具体取决于系统配置）。

特点：是最常用的时钟源，兼顾频率稳定性和外设兼容性，适用于大多数常规定时场景（如本实验中的 1 秒周期计时）。

2. GPTIMER_CLK_SRC_XTAL（外部晶振时钟）

对应时钟：外部 40MHz 晶振（XTAL）。

频率：固定为 40MHz。

特点：由外部硬件晶振提供，频率稳定性高（受温度、电压影响小），适合对时钟精度要求较高的场景（如高精度计时、频率测量等）。

3. GPTIMER_CLK_SRC_PLL_F160M（PLL 产生的 160MHz 时钟）

对应时钟：由 PLL（锁相环）倍频产生的 160MHz 时钟。

频率：固定为 160MHz。

特点：属于高速时钟源，适合需要更高计数频率的场景（如高频脉冲计数、短周期定时等），但频率稳定性略低于外部晶振。

4. GPTIMER_CLK_SRC_RTC8M（RTC 模块 8MHz 时钟）

对应时钟：RTC（实时时钟）模块内置的 8MHz 时钟。

频率：约 8MHz（由内部 RC 振荡器产生，精度较低）。

特点：功耗较低，适合低功耗场景（如设备处于浅睡眠时的定时），但精度较差（受温度影响较大）。

1.3 实验中使用的时钟源

在提供的工程代码（`components/counter/counter.c`）中，通用定时器的时钟源配置为 `GPTIMER_CLK_SRC_DEFAULT`：

```
gptimer_config_t gptimer_cfg={
    .clk_src=GPTIMER_CLK_SRC_DEFAULT, // 使用默认时钟源（APB_CLK, 80MHz）
    // 其他配置...
};
```

该配置通过 `APB_CLK` 实现 1MHz 的计数分辨率（`resolution_hz=1000000`），并结合 1000000 的闹钟值，实现了 1 秒周期的定时中断，满足数字时钟的基础计时需求。

2 通用定时器相关 API

2.1 创建和启动定时器

```
gptimer_handle_t gptimer = NULL;
gptimer_config_t timer_config = {
    .clk_src = GPTIMER_CLK_SRC_DEFAULT, // 选择默认的时钟源
```

```

    .direction = GPTIMER_COUNT_UP,      // 计数方向为向上计数
    .resolution_hz = 1 * 1000 * 1000,    // 分辨率为 1 MHz, 即 1 次滴答为 1 微秒
};
// 创建定时器实例
gptimer_new_timer(&timer_config, &gptimer);
// 使能定时器
gptimer_enable(gptimer);
// 启动定时器
gptimer_start(gptimer);

```

当创建定时器实例时, 我们需要通过 `gptimer_config_t` 配置时钟源、计数方向和分辨率等参数。这些参数将决定定时器的工作方式。然后调用 `gptimer_new_timer()` 函数创建一个新的定时器实例, 该函数将返回一个指向新实例的句柄。定时器的句柄实际上是一个指向定时器内存对象的指针, 类型为 `gptimer_handle_t`。

以下是 `gptimer_config_t` 结构体的其他配置参数及其解释:

`gptimer_config_t::clk_src` 选择定时器的时钟源。可用时钟源列在 `gptimer_clock_source_t` 中, 只能选择其中一个。不同的时钟源会在分辨率, 精度和功耗上有所不同。

`gptimer_config_t::direction` 设置定时器的计数方向。支持的方向列在 `gptimer_count_direction_t` 中, 只能选择其中一个。

`gptimer_config_t::resolution_hz` 设置内部计数器的分辨率。计数器每滴答一次相当于 $1 / \text{resolution_hz}$ 秒。

`gptimer_config_t::intr_priority` 设置中断的优先级。如果设置为 0, 则会分配一个默认优先级的中断, 否则会使用指定的优先级。

定时器在启动前必须要先使能, 使能函数 `gptimer_enable()` 可以将驱动的内部状态机切换到激活状态, 这里面还会包括一些系统性服务的申请/注册等工作, 如申请电源管理锁。

`gptimer_start()` 函数用于启动定时器。启动后, 定时器将开始计数, 并在计数到达最大值或者最小值时 (取决于计数方向) 自动溢出, 从 0 开始重新计数。

2.2 触发周期性警报事件

```

gptimer_alarm_config_t alarm_config = {
    .reload_count = 0,      // 当警报事件发生时, 定时器会自动重载到 0
    .alarm_count = 1000000, // 设置实际的警报周期, 因为分辨率是 1us, 所以 1000000 代表 1s
    .flags.auto_reload_on_alarm = true, // 使能自动重载功能
};
// 设置定时器的警报动作
gptimer_set_alarm_action(gptimer, &alarm_config);

```

以下是 `gptimer_alarm_config_t` 结构体中的每个必要成员项的作用, 通过配置这些参数, 用户可以灵活地控制定时器的警报行为, 以满足不同的应用需求。

`gptimer_alarm_config_t::alarm_count` 设置触发警报事件的目标计数值。当定时器计数值达到该值时，将触发警报事件。设置警报值的时候也需要考虑定时器的计数方向，如果当前计数值已经 越过 了警报值，那么警报事件会立刻触发。

`gptimer_alarm_config_t::reload_count` 设置警报事件发生时要自动重载的计数值。此配置仅在 `gptimer_alarm_config_t::flags::auto_reload_on_alarm` 标志为 `true` 时生效。实际的警报周期将会由 `|alarm_count - reload_count|` 决定。从实际应用触发，不建议将警报周期设置成小于 `5us`。

```
gptimer_event_callbacks_t cbs = {
    .on_alarm = example_timer_on_alarm_cb, // 当警报事件发生时，调用用户回调函数
};
// 注册定时器事件回调函数，允许携带用户上下文
gptimer_register_event_callbacks(gptimer, &cbs, NULL);
```

`gptimer_register_event_callbacks()` 函数用于注册定时器事件的回调函数。当定时器触发特定事件（如警报事件）时，将调用用户定义的回调函数。用户可以在回调函数中执行自定义操作，例如发送信号，从而实现更灵活的事件处理机制。由于回调函数是在中断上下文中执行的，因此在回调函数中应该避免执行复杂的操作（包括任何可能导致阻塞的操作），以免影响系统的实时性。

```
bool TimerCallback (gptimer_handle_t timer, const gptimer_alarm_event_data_t *edata,
void *user_ctx)

{

    // 处理事件回调的一般流程

    return 0;

}
```

返回值 `bool` 表示回调执行后是否需要继续触发定时器。通常：

返回 `true`：表示继续运行定时器（适用于周期性定时）；

返回 `false`：表示停止定时器（适用于单次定时）。

参数解析

`gptimer_handle_t timer`：定时器句柄（标识当前触发回调的定时器实例）。通过该句柄可操作定时器（如动态修改定时参数）。

`const gptimer_alarm_event_data_t *edata`：指向闹钟事件数据的指针，包含定时器触发时的详细信息（如 `timestamp` 字段表示触发时的定时器计数值）。

`void *user_ctx`：用户上下文指针，是注册回调时传入的自定义数据（如全局变量、结构体等），用于在回调中传递额外信息。

LEDC PWM 介绍与 API

1 LED 控制器 (LEDC) 概述

LED 控制器 (LEDC) 是 ESP32 的重要外设，主要用于控制 LED，也可产生 PWM 信号用于其他设备的控制。在本实验中，我们使用 LEDC 生成 PWM 脉冲信号来控制电机转速。

LEDC 通道共有两组，分别为 8 路高速通道和 8 路低速通道。高速通道模式在硬件中实现，可以自动且无干扰地改变 PWM 占空比。低速通道模式下，PWM 占空比需要由软件中的驱动器改变。

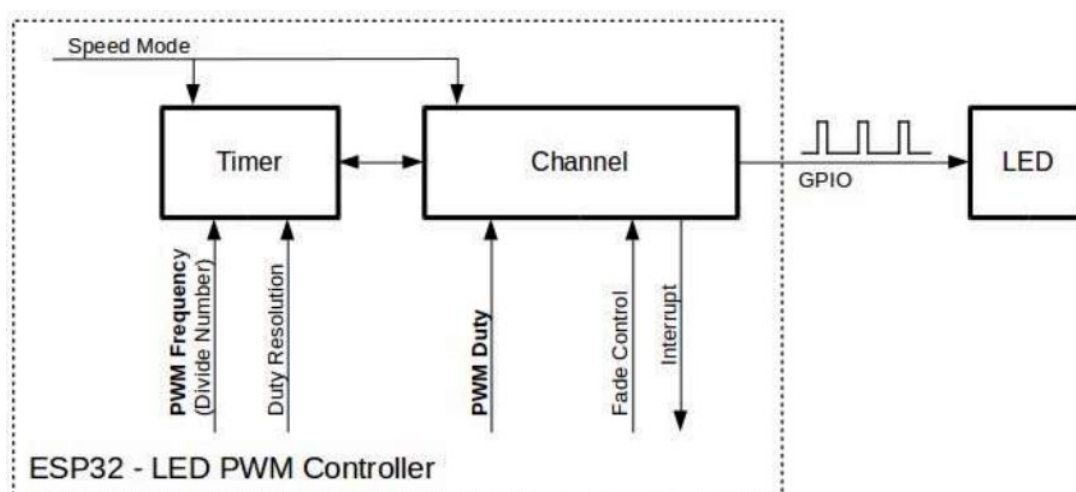


图 15

2 LEDC 工作模式与时钟源

2.1 高速模式与低速模式

LEDC 通道分为高速通道和低速通道两组。高速通道在硬件层面实现 PWM 控制，可以无干扰地自动改变占空比；而低速通道需要通过软件驱动程序来调整 PWM 参数。

2.2 时钟源特性

ESP32 LEDC 支持多种时钟源，具体特性如图 16。

ESP32 LEDC 时钟源特性

时钟名称	时钟频率	速度模式	时钟功能
APB_CLK	80 MHz	高速 / 低速	/
REF_TICK	1 MHz	高速 / 低速	支持动态调频 (DFS) 功能
RC_FAST_CLK	~ 8 MHz	低速	支持动态调频 (DFS) 功能，支持 Light-sleep 模式

图 16

3 LEDC 配置流程

首先需要配置 LEDC 定时器，设置 PWM 信号的频率和占空比分辨率：

```
ledc_timer_config_t ledc_timer = {
    .speed_mode = LEDC_MODE,           // 速度模式
    .duty_resolution = LEDC_DUTY_RES, // 占空比分辨率
    .timer_num = LEDC_TIMER,           // 定时器索引
    .freq_hz = LEDC_FREQUENCY,         // PWM 频率
    .clk_cfg = LEDC_AUTO_CLK           // 时钟源配置
};
ledc_timer_config(&ledc_timer);
```

接着配置 LEDC 通道，将定时器与 GPIO 输出引脚绑定：

```
ledc_channel_config_t ledc_channel = {
    .speed_mode = LEDC_MODE,           // 速度模式
    .channel = LEDC_CHANNEL,           // 通道编号
    .timer_sel = LEDC_TIMER,           // 绑定的定时器
    .intr_type = LEDC_INTR_DISABLE,    // 中断类型
    .gpio_num = PWM_PIN,               // 输出 GPIO 引脚
    .duty = LEDC_DUTY,                 // 初始占空比
    .hpoint = 0                        // 高电平起始点
};
ledc_channel_config(&ledc_channel);
```

在本实验中，我们采用低速模式（脉冲频率低于 16kHz），使用其动态调频（DFS）功能改变 PWM 的频率，使用固定的占空比，所有相关参数的宏定义在 `stepper_motor.h` 中。

4 LEDC 相关 API

1. 频率设置函数

```
esp_err_t ledc_set_freq(ledc_mode_t speed_mode,
                        ledc_timer_t timer_num,
                        uint32_t freq_hz)
```

- 功能：设置 LEDC 通道的 PWM 频率
- 参数：速度模式、定时器索引、目标频率
- 返回值：成功返回 ESP_OK，失败返回错误码

2. 占空比设置函数

```
esp_err_t ledc_set_duty(ledc_mode_t speed_mode,
                       ledc_channel_t channel,
                       uint32_t duty)
```

- 功能：设置 LEDC 通道的 PWM 占空比
- 参数：速度模式、通道编号、占空比值
- 注意：需要调用 `ledc_update_duty()` 使设置生效

3. 占空比更新函数

```
esp_err_t ledc_update_duty(ledc_mode_t speed_mode,  
                           ledc_channel_t channel)
```

- 功能：激活最新的 PWM 占空比设置
- 说明：新的 PWM 参数在下个 PWM 周期生效